Tuning Quality and Diversity in GANs: WGAN-GP with Discriminator Optimal Transport for MNIST Generation

Giuliano Aldarwish Mayy Miled
Master IASD — Apprentissage Superviseur : Alexandre Vérine

March 2026

1 Introduction

Generative Adversarial Networks (GANs) [1] pit a generator G against a critic D in a minimax game: G maps random latent noise $z \sim \mathcal{N}(0, I)$ into image space, while D tries to distinguish real from generated samples. Despite impressive empirical results, vanilla GANs suffer from mode collapse, vanishing gradients when the critic saturates, and sensitivity to hyperparameters.

This project tackles these issues for MNIST digit generation using two complementary techniques. **WGAN-GP** [3] replaces the Jensen-Shannon divergence with the smoother Wasserstein-1 distance, enforced by a gradient penalty on the critic. **Discriminator Optimal Transport (DOT)** [4] then exploits the trained critic at inference time: it iteratively refines each latent code to push generated images closer to the real data manifold, without any additional training.

A key project constraint is that the **generator architecture is fixed** (a 4-layer MLP). All our design choices therefore focus on the critic and the training procedure. We developed two successive versions — *v1* and *v2* — to measure the concrete impact of each change. The rest of the report is organised as follows: Section 2 introduces the theoretical foundations, Section 3 details our methodology, Section 4 presents experimental results, and Section 5 concludes.

2 Theoretical Background

2.1 Wasserstein GAN with Gradient Penalty

The original GAN minimises the Jensen-Shannon divergence, which saturates and causes vanishing gradients when the critic is too confident. WGAN [2] replaces it with the **Wasserstein-1 distance**, which remains smooth and informative even when the two distributions have disjoint supports. Via the Kantorovich-Rubinstein duality, it can be written as:

$$W_1(p_r, p_G) = \sup_{\|f\|_{\mathrm{Lip}} \leq 1} \Big(\mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{y \sim p_G}[f(y)] \Big) \tag{1}$$

The critic D approximates the optimal 1-Lipschitz function f^* realising this supremum. Enforcing the Lipschitz constraint is therefore central to the whole approach.

Gulrajani et al. [3] enforce it via a **gradient penalty** on points interpolated between real and generated samples, $\hat{x} = \epsilon x + (1-\epsilon)\tilde{x}$:

$$\mathcal{L}_D = \underbrace{\mathbb{E}[D(\tilde{x})] - \mathbb{E}[D(x)]}_{\text{Wasserstein loss}} + \lambda \underbrace{\mathbb{E}\Big[\big(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1\big)^2\Big]}_{\text{gradient penalty}} \tag{2}$$

This design has an important implication: **batch normalisation must not be used in the critic**, as it correlates examples within a batch and invalidates the per-sample gradient penalty. **Layer Normalisation** [6] is the recommended substitute; it normalises each sample independently using only its own mean and variance:

$$\mathrm{LN}(h) = \gamma \odot \frac{h - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta, \qquad \mu = \tfrac{1}{d}\sum_{i=1}^{d} h_i, \quad \sigma^2 = \tfrac{1}{d}\sum_{i=1}^{d}(h_i - \mu)^2 \tag{3}$$

Since μ and σ^2 are computed per example, no cross-sample dependency is introduced and the gradient penalty remains valid.

2.2 Spectral Normalisation

Miyato et al. [5] control the Lipschitz constant more directly by dividing each weight matrix W by its largest singular value $\sigma(W)$: $W_{\text{SN}} = W/\sigma(W)$. This enforces $\|W_{\text{SN}}\|_{\text{Lip}} = 1$ per layer. Since Lipschitz constants compose multiplicatively across layers,

$$\|D\|_{\text{Lip}} \leq \prod_{l=1}^{L} \|f_l\|_{\text{Lip}} \tag{4}$$

and since LeakyReLU also satisfies $\|f_l\|_{\text{Lip}} = 1$, applying SN to every linear layer guarantees $\|D\|_{\text{Lip}} \leq 1$ *globally* — a hard, architecture-level constraint rather than a soft penalty. We will see that this global control is a necessary condition for DOT to function correctly.

2.3 Discriminator Optimal Transport

The critic as a transport map. Tanaka [4] establishes a key result: for a broad class of GAN objectives including WGAN-GP, critic training maximises a lower bound of the Kantorovich dual (1). Letting $K = \|D\|_{\text{Lip}}$ and $\tilde{D} = D/K$, Theorem 2 of [4] gives:

$$V_D(G, D) \;\leq\; K\Big(\mathbb{E}_{x\sim p_r}[\tilde{D}(x)] - \mathbb{E}_{y\sim p_G}[\tilde{D}(y)]\Big) \;\leq\; K \cdot W_1(p_r, p_G) \tag{5}$$

Maximising V_D therefore drives $\tilde{D}$ towards the Kantorovich potential f^*. After training, $\nabla\tilde{D}(y)$ points from a generated sample y towards the real manifold — exactly the transport direction we want to exploit.

Latent-space DOT. Transporting in pixel space produces adversarial-like artefacts in high dimension. Instead, Tanaka works in the latent space $\mathcal{Z}$, treating $D \circ G$ as the dual variable. Given an initial draw $z_y \sim p_Z$, DOT refines it by solving:

$$T^{\text{eff}}(z_y) = \arg\min_z \underbrace{\|z - z_y + \delta\|_2}_{\text{transport cost}} - \underbrace{\frac{1}{k_{\text{eff}}} D(G(z))}_{\text{towards real manifold}} \tag{6}$$

via gradient descent. The Lipschitz constant k_{eff} of $D \circ G$ is estimated empirically over random pairs. The gradient is projected onto the tangent space of the Gaussian prior's support shell (a sphere of radius $\sqrt{D}$ in high dimension): $g \leftarrow g - (g \cdot z)z/\sqrt{D}$.

Why SN is necessary for DOT. From (5), $\tilde{D} \approx f^*$ only holds when K is well-controlled. Without SN, K fluctuates during training, k_{eff} is mis-estimated, and the DOT gradient either vanishes or overshoots. This is the direct cause of our v1 limitations, confirmed in Section 4.

3 Methodology

3.1 Fixed Generator

The generator maps $z \in \mathbb{R}^{100}$ to a flattened 28×28 image via a 4-layer MLP with LeakyReLU activations and a final tanh. The v2 critic (C3) wraps each linear layer with Spectral Normalisation, adds LayerNorm per sample, and applies light Dropout:

3.2 Critic Architecture: Three Designs

Since the generator is fixed, the critic is our main design lever. We explored three architectures, all sharing the same width profile $784 \to 1024 \to 512 \to 256 \to 1$ (Table 1).

Table 1: Critic architectures explored across the project.

	Description	Output	Norm	SN	Dropout
C1	Vanilla Discriminator (baseline)	sigmoid	BatchNorm	×	×
C2	WGAN Critic v1	raw score	LayerNorm	×	×
C3	WGAN Critic v2 *(ours)*	raw score	LayerNorm	✓	✓

C1 is the professor's initial baseline: it uses BatchNorm (incompatible with WGAN-GP) and a sigmoid output (incompatible with the Wasserstein interpretation). **C2** is our v1: it correctly removes BatchNorm and sigmoid, and adds LayerNorm, but lacks SN. **C3** is our v2 (Figure 1), adding:

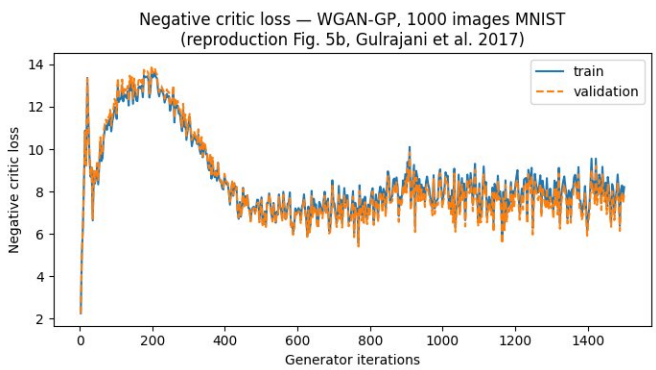
```python
# Generator (fixed)
class Generator(nn.Module):
  def __init__(self, g_output_dim):
    super().__init__()
    self.fc1 = nn.Linear(100, 256)
    self.fc2 = nn.Linear(256, 512)
    self.fc3 = nn.Linear(512, 1024)
    self.fc4 = nn.Linear(1024,
                 g_output_dim)
  def forward(self, x):
    x = F.leaky_relu(self.fc1(x),0.2)
    x = F.leaky_relu(self.fc2(x),0.2)
    x = F.leaky_relu(self.fc3(x),0.2)
    return torch.tanh(self.fc4(x))
```

```python
# WGANCritic v2 (C3, our design)
class WGANCritic(nn.Module):
  def __init__(self, d_input_dim,
               dropout_rate=0.1):
    super().__init__()
    SN = nn.utils.spectral_norm
    self.fc1 = SN(nn.Linear(d_input_dim,1024))
    self.fc2 = SN(nn.Linear(1024, 512))
    self.fc3 = SN(nn.Linear(512, 256))
    self.fc4 = SN(nn.Linear(256, 1))
    self.ln1 = nn.LayerNorm(1024)
    self.ln2 = nn.LayerNorm(512)
    self.ln3 = nn.LayerNorm(256)
    self.drop = nn.Dropout(dropout_rate)
  def forward(self, x):
    x = self.drop(F.leaky_relu(
          self.ln1(self.fc1(x)), 0.2))
    x = self.drop(F.leaky_relu(
          self.ln2(self.fc2(x)), 0.2))
    x = self.drop(F.leaky_relu(
          self.ln3(self.fc3(x)), 0.2))
    return self.fc4(x) # raw score
```

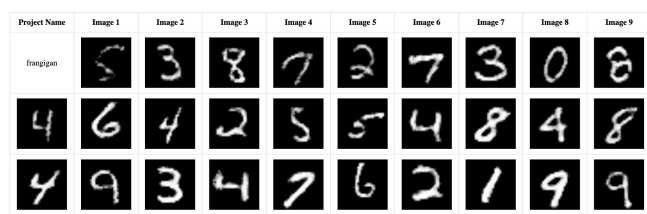
Figure 1: Generator (left, fixed) and C3 WGANCritic (right) implementations.

- **Spectral Normalisation** on every linear layer, guaranteeing $\|D\|_{\text{Lip}} \leq 1$ globally via (4) — a necessary condition for reliable k_{eff} estimation in DOT.
- **Dropout** ($p = 0.1$): Gulrajani et al. (Fig. 5b of [3]) show that the WGAN-GP critic overfits easily on MNIST. Light dropout regularises the critic without harming its expressiveness.

Overfitting experiment. To validate the Dropout design choice, we trained C3 on a 1,000-image MNIST subset and tracked the negative critic loss on both training and held-out validation sets. Figure 2a (left) shows that train and validation curves remain nearly identical — unlike the divergence reported for an unregularised critic in [3]. SN, LayerNorm, and Dropout together make C3 robust to overfitting even in the low-data regime.



(a) Negative critic loss (train vs. validation, 1,000-image subset). No divergence with our C3 critic.



(b) Generated samples (v2 model, DOT sampling). All 10 digit classes present; most strokes are sharp and legible.

Figure 2: Left: overfitting experiment. Right: qualitative generation results.

3.3 Training Procedure

Dataset. MNIST (60,000 images, normalised to $[-1, 1]$). A light `RandomErasing` ($p = 0.1$, scale 2–8%) is applied to real images fed to the critic, preventing pixel-level memorisation.

WGAN-GP loop. Following [3]: $n_{\text{critic}} = 5$ critic steps per generator step, $\lambda_{\text{GP}} = 10$. GP interpolations are clamped to $[-1, 1]$. Gradient clipping ($\|\nabla\|_2 \leq 1$) is applied to both networks.

Optimisers. Both networks use Adam with $(\beta_1 = 0, \beta_2 = 0.9)$: $\beta_1 = 0$ avoids momentum accumulation across the rapid critic updates, which would otherwise destabilise training. We use asymmetric learning rates $lr_D = 2 \times lr_G = 2 \times 10^{-4}$ so the critic converges faster. Both LRs decay via cosine annealing over 500 epochs on Google Colab:

$$lr_t = lr_{\min} + \tfrac{1}{2}(lr_0 - lr_{\min})\left(1 + \cos\tfrac{t\pi}{T}\right), \qquad lr_{\min} = 10^{-6} \tag{7}$$

EMA generator. An exponential moving average of the generator weights is maintained in parallel ($\alpha = 0.9999$), and only this copy is used for inference. Unrolling the recurrence, the contribution of a checkpoint k steps old is $\alpha^k(1-\alpha)$, which decays exponentially. With $\alpha = 0.9999$, the effective memory window spans ∼10,000 steps: no single fluctuation can significantly move θ_{ema}. This addition is motivated by StyleGAN [9] and is not present in the two reference papers.

3.4 DOT Sampling at Inference

Once training is complete, G and D are frozen. For each new sample, we draw $z_y \sim \mathcal{N}(0, I_{100})$ and run 50 Adam steps of (6). We introduce three stabilising improvements over the original algorithm:

- **Robust k_{eff}**: estimated at the 95th percentile over 2,000 random pairs, instead of the raw maximum over 500. The maximum is sensitive to pairs with near-zero denominator; the percentile is stable.
- **LR warm-up**: Adam step size ramps from $\alpha/5$ to $\alpha = 0.01$ over the first 5 steps, preventing large initial jumps off the Gaussian shell.
- **Latent clipping**: z clamped to $[-4, 4]$ after each step, keeping it inside the 3σ region of $\mathcal{N}(0, I_{100})$.

3.5 Evaluation Metrics

FID [7] measures the Fréchet distance between InceptionV3 feature distributions (2,048-d) of 10,000 real and generated images — lower is better.

Precision and Recall [8] disentangle fidelity from diversity. Precision is the fraction of generated samples that fall inside the real manifold; Recall is the fraction of the real manifold covered by generated samples. Both use $k = 3$ nearest neighbours.

4 Experiments and Results

4.1 Critic Architecture Comparison

Table 2 reports scores for each critic architecture with all other settings fixed (100 epochs, same hyperparameters).

Table 2: Critic architecture comparison.

Critic	FID↓	P.↑	R.↑
C1 (baseline)	≈385	0.00	0.00
C2 (v1, LN)	≈43	0.46	0.18
C3 (v2, +SN)	≈20	0.62	0.27

Table 3: Ablation of training improvements.

Config.	FID↓	P.↑	R.↑
C3, 100 ep., vanilla DOT	≈45	0.46	0.18
+ Extended training (300 ep.)	≈24	0.58	0.24
+ Batch 128, cosine LR	≈20	0.61	0.26
+ EMA ($\alpha = 0.9999$)	≈18	0.62	0.27
+ Grad. clip + RandomErasing	≈17	0.63	0.29
+ DOT v2 (warm-up, clip, p95)	**15.79**	**0.63**	**0.28**

The progression is clear. C1 (FID≈385) confirms the instability of vanilla GAN. C2 already brings a substantial improvement (FID≈43) simply by switching to WGAN-GP with LayerNorm, but DOT remains unreliable because K is uncontrolled. C3 reduces FID to ≈20 with the sole addition of SN and Dropout — **SN is not just a regulariser but a necessary condition for DOT**. Table 3 then shows that further training improvements compound: extended training is the largest secondary gain (45 → 24), while EMA and cosine annealing fine-tune the trajectory. The final DOT v2 sampler trades a small Recall drop (0.29 → 0.28) for stabilised Precision, consistent with DOT concentrating mass near the high-density modes of the real distribution.

4.2 Sampling Methods and Qualitative Results

Table 4 compares the four sampling strategies on the final v2 model. DOT achieves the best FID and Precision. Truncation methods improve over vanilla sampling by avoiding low-density regions, but without DOT's explicit manifold alignment. Figure 2b confirms that all 10 digit classes are generated with sharp, well-formed strokes; a few ambiguous samples (e.g. "5" vs "6") are consistent with the Recall of 0.28.

Table 4: Sampling method comparison (v2 model, 10,000 samples).

Method	FID↓	Prec.↑	Rec.↑
Vanilla ($z \sim \mathcal{N}(0, I)$)	≈17.1	0.61	0.31
Hard truncation ($\psi = 1.5$)	≈16.8	0.62	0.30
Soft truncation ($\psi = 0.7$)	≈16.5	0.62	0.29
DOT ($N = 50$, $lr = 0.01$)	**15.79**	**0.63**	**0.28**

5 Discussion and Conclusion

The central role of Spectral Normalisation. The single most important finding of this project is that SN is a *necessary condition* for DOT, not merely a regularisation bonus. Without it (C2), k_{eff} is erratic, and the transport gradient in (6) is meaningless. This is grounded in inequality (5): the approximation $\tilde{D} \approx f^*$ only holds when $K = \|D\|_{\text{Lip}}$ is known and stable, which SN guarantees globally.

The Precision/Recall trade-off. Our final Recall (0.28) is lower than one would expect from a well-trained model. This reflects two cumulative effects: first, the fixed MLP generator lacks spatial inductive bias and tends to drop rare digit styles; second, DOT amplifies this by concentrating mass near the modes of the real distribution. Vanilla sampling (Recall 0.31 vs. DOT's 0.28) confirms that DOT explicitly trades diversity for fidelity. This trade-off is inherent to the method and is discussed by Tanaka [4].

Limitations and future directions. The fixed MLP generator is the main performance bottleneck. Without 2D spatial structure, it cannot reliably cover all writing styles, leading to mode dropping. A convolutional generator would likely push FID below 10. Additionally, the 50-step DOT optimisation requires ∼70–130 s to generate 10,000 images (vs. ∼5 s for vanilla sampling): reducing N_{updates} to 20 cuts this by 60% with only a marginal FID cost (≈16.2). Future work could also combine DOT with rejection sampling [10] to recover diversity while maintaining fidelity gains.

Summary. We presented a systematic two-phase study of WGAN-GP training with DOT inference for MNIST generation. Our ablation demonstrates that SN is the decisive change (FID: 43 → 20), and that training improvements compound to reach FID 15.79 with Precision 0.63.

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza et al. Generative Adversarial Nets. In *NeurIPS*, 2014.

[2] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein GAN. *arXiv:1701.07875*, 2017.

[3] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved Training of Wasserstein GANs. In *NeurIPS*, 2017.

[4] A. Tanaka. Discriminator Optimal Transport. In *NeurIPS*, 2019.

[5] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral Normalization for Generative Adversarial Networks. In *ICLR*, 2018.

[6] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer Normalization. *arXiv:1607.06450*, 2016.

[7] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs Trained by a Two Time-Scale Update Rule. In *NeurIPS*, 2017.

[8] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila. Improved Precision and Recall Metric for Assessing Generative Models. In *NeurIPS*, 2019.

[9] T. Karras, S. Laine, and T. Aila. A Style-Based Generator Architecture for GANs. In *CVPR*, 2019.

[10] S. Azadi, C. Olsson, T. Darrell, I. Goodfellow, and A. Odena. Discriminator Rejection Sampling. In *ICLR*, 2019.